Efficient Signed-Frobenius Orbit Canonicalization and Phase-Aware Representations for Elliptic-Curve Index Calculus

Anonymous

Draft of October 14, 2026

Abstract

On a Koblitz curve over $\mathbb{F}_{2^n}$ the Frobenius endomorphism τ and negation generate a group Γ of order $2n$ acting on the prime-order subgroup, and an index calculus with a Γ-invariant factor base needs $2n$ times fewer unknowns. That reduction is known (Galbraith, Granger, Merz and Petit, SAC 2020). What it costs to realize is not: every decomposed point has to be mapped to its orbit representative together with its sign and Frobenius phase k, the tables the pipeline probes have to be keyed by representatives, and the phase has to survive from the decomposition stage, which sees only x-coordinates, to the relation matrix, which needs λ^k. This paper treats that as an algorithmic problem. We define the signed-Frobenius representation $P = \epsilon\,\tau^{-k}\hat{P}$ and prove that orbits in the prime subgroup are always full when n is prime, so the only exceptional words are $x \in \mathbb{F}_2$. We give an exact shortcut for the normal-basis least-rotation canonical form, built on the standard fact that the least rotation of a binary necklace begins at a longest run of zeros, prove it bit-identical to the exhaustive scan, and measure it at 4.78 rotations per 83-bit word against 82. We describe phase-carrying layouts: an 18-byte root record with a 7-bit phase, a 2.3× smaller index than an open-addressed table; an orbit tag that makes summand recovery in a folded pair table $O(n)$ instead of $O(|F|)$; and the rule that canonical keys must never be indexed by their leading bits. We then report what survives complete relation collection, with every logarithm verified: a Γ-orbit Gröbner base solves a complete ECDLP at $n = 19$ in 6.85× less CPU than a plain base; normal-basis canonicalization makes the signed-Frobenius Pollard rho reference 4.1× cheaper in retired instructions, which reverses an apparent index-calculus win; and orbit folding of the pair table divides stored entries by $2n$ while moving total operations, in one four-width sweep, by +2.7% at one width and −12% at another. Nothing here beats Pollard rho on equal terms. With its setup charged, the phase-aware index calculus is slower than the matched rho at every degree priced (1.3–331×). With the setup excluded it is 8.8–17.5× faster online on one target at $r \le 2^{47}$, but the ledger's model puts Bernstein–Lange rho, given the same precomputation, 2–8× faster still. The contribution is the representation, its proofs, its measured costs, and the discipline that keeps a constant-factor gain in one stage from being reported as an attack.

Contents

1 Introduction **3**

2 Preliminaries **4**

2.1 Koblitz curves and the Frobenius endomorphism 4

2.2 The signed-Frobenius group 4

2.3 Normal bases and rotation 5

2.4 Index calculus with a factor base of orbits 5

3 The signed-Frobenius representation **6**
3.1 Definition . . . 6
3.2 A Frobenius-invariant sign chooser . . . 6
3.3 Phase-aware relations . . . 6

4 Canonicalization by cyclic zero gaps **7**
4.1 The lemma . . . 7
4.2 Two implementations . . . 8
4.3 Correctness, including the exceptional words . . . 9

5 Phase-aware memory layouts **10**
5.1 Representatives on disk . . . 10
5.2 The packed root record . . . 10
5.3 Canonical keys are skewed . . . 11
5.4 The folded pair table and the orbit tag . . . 11

6 Carrying the phase through a pipeline **12**
6.1 One convention per pipeline . . . 12
6.2 The index-calculus data flow (`crypto`, $n \leq 63$) . . . 13
6.3 The matched reference: signed-Frobenius Pollard rho . . . 13
6.4 The $n = 83$ root index (`cryptanalysis`) . . . 13
6.5 The $n = 19$ Sage pipeline (`cryptanalysis`) . . . 14

7 Benchmarks **14**
7.1 How the numbers are read . . . 14
7.2 Canonicalization cost . . . 14
7.3 The canonicalization inside the reference . . . 16
7.4 Orbit compression of tables . . . 17
7.5 End to end, with verified logarithms . . . 18
7.6 What survives complete relation collection . . . 20

8 Related work and what is not new here **20**

9 Limitations and claims not made **22**

10 Conclusion **23**

A Reproduction **26**

1 Introduction

A Koblitz curve $E_a : y^2 + xy = x^3 + ax^2 + 1$ is defined over $\mathbb{F}_2$ and used over $\mathbb{F}_{2^n}$, so the 2-power Frobenius $\tau(x, y) = (x^2, y^2)$ is an endomorphism that acts on the prime-order subgroup G as multiplication by a known scalar λ. Together with negation it generates a group Γ of order $2n$, and every algorithm for the discrete logarithm in G can work on Γ-orbits instead of points. For Pollard rho that is worth $\sqrt{2n}$ [16, 26]; for index calculus with a Γ-invariant factor base it is worth $2n$ in the number of relations and $(2n)^2$ in the linear algebra [14]. Both facts are classical and neither is claimed here.

What is claimed is narrower and, we think, more useful to someone who has to build the thing. The $2n$ saving is a statement about counts. To collect it, a pipeline has to map every decomposed point P to its orbit representative $\hat{P}$ *and* to the sign ϵ and phase k with $P = \epsilon\tau^{-k}\hat{P}$, because the relation coefficient is $\epsilon\lambda^{-k}$ and not 1. It has to do this for every candidate the decomposition produces, not only the ones that turn into relations, since the candidates are what the tables are probed with. The tables have to be keyed by representatives, so that they shrink by $2n$ too, and a hit in such a table has to give back the actual summands, which means recovering a phase that was never stored. And at the end every logarithm has to verify, which it does only if nothing was lost on the way. Each of these steps has a cost, and in the implementations we measured that cost has at various times been the majority of the pipeline, the majority of the *reference* the pipeline was compared with, and the difference between a reported win and a reported loss.

Contributions.

1. *A representation with proofs (Sections 2–3).* The signed-Frobenius form $\mathrm{can}(P) = (\hat{P}, \epsilon, k)$; a proof that Γ acts freely on $G\backslash\{O\}$ whenever $r \nmid \#E_a(\mathbb{F}_{2^d})$ for proper divisors $d \mid n$, so that for prime n the only exceptional words a canonicalizer meets are $x \in \mathbb{F}_2$; a Frobenius-invariant sign chooser $\mathrm{Tr}(y/x)$ for odd n; and the reconstruction theorem that makes phase-aware relation collection correct.

2. *An exact canonicalization shortcut (Section 4).* It rests on the standard fact that the least rotation of a binary necklace begins at a longest cyclic zero run, used here as a candidate filter. Two implementations (bit-parallel for one-word fields, set-bit scan for two-limb fields) are proved bit-identical to the exhaustive scan, including on the exceptional words, and measured at 4.78 rotations per 83-bit word against 82.

3. *Phase-aware layouts (Section 5).* The 18-byte root record with an exact 83-bit key and a 7-bit phase; a hashed-bucket directory with singleton fingerprints; the orbit tag that makes recovery from a folded pair table $O(n)$; the row filter that stores each orbit of sums once; and the observation that canonical keys are skewed toward small values and must be hashed before they are indexed, a mistake two independent implementations made.

4. *Benchmarks that say what class they are (Sections 6–7).* Every number is footnoted to a committed artefact, read in one unit against a matched signed-Frobenius rho and a generic floor, labelled engineering, relabelling or accounting, and marked as isolated or exploratory. The end-to-end rows all have verified logarithms. None is an advance over rho.

What the reader should take away. Three things. The normal-basis least rotation, not the squaring chain, is the right canonicalization, by 8–15× per key and 4× on the whole rho process; and because a signed-Frobenius rho spends half its instructions canonicalizing, any comparison against rho that does not match the canonicalization is measuring the wrong thing (Section 7.3). The counts a fold divides by $2n$ are not the counts the cost is proportional to: at $n = 61$ the folded pair table stores 121× fewer pairs and costs 1.2% more operations in the run

that made it the default, and in a separate four-width sweep it is 2.7% dearer than the full tier at one width and 12% cheaper at another (Section 7.4). And the one place a phase-aware base makes a complete index calculus faster with verified answers, 6.85× at $n = 19$, is a toy whose base does not exist at $n = 131$ (Sections 7.5, 8).

Organisation. Section 2 fixes notation and proves the free-action lemma. Section 3 defines the representation and proves transport and reconstruction. Section 4 gives the zero-gap lemma, the algorithms and their correctness. Section 5 describes the layouts. Section 6 fixes where the phase lives in each pipeline. Section 7 reports the measurements. Section 8 says what is prior art, and Section 9 what is not claimed. Appendix A lists the artefacts.

2 Preliminaries

2.1 Koblitz curves and the Frobenius endomorphism

Let n be an odd prime and $K = \mathbb{F}_{2^n}$. The two Koblitz (anomalous binary) curves are

$$E_a : \; y^2 + xy = x^3 + ax^2 + 1, \qquad a \in \{0, 1\},$$

defined over $\mathbb{F}_2$ [21]. The 2-power Frobenius $\tau(x, y) = (x^2, y^2)$ is an endomorphism of E_a satisfying

$$\tau^2 - \mu\tau + 2 = 0, \qquad \mu = (-1)^{1-a},$$

and τ^n is the identity on $E_a(K)$. Negation is $-(x, y) = (x,\, x + y)$, so the only points with $-P = P$ are O and the 2-torsion point $(0, 1)$. Throughout, $G \leq E_a(K)$ denotes a cyclic subgroup of prime order r with $r > 4$; for the fields in this paper the cofactor $\#E_a(K)/r$ is 4 (for $a = 0$) or 2 (for $a = 1$).

Because τ commutes with scalar multiplication it preserves G, and because G is cyclic of prime order it acts on G as a scalar: there is a unique $\lambda \in (\mathbb{Z}/r\mathbb{Z})^\times$ with $\tau(P) = [\lambda]P$ for all $P \in G$. Reducing the characteristic equation, $\lambda^2 - \mu\lambda + 2 \equiv 0 \pmod r$ and $\lambda^n \equiv 1 \pmod r$. The scalar λ is the quantity that lets a Frobenius image be folded into a logarithm: $\log_P(\tau^k Q) = \lambda^k \log_P Q$.

2.2 The signed-Frobenius group

Let $\Gamma = \langle \tau, -1 \rangle$ be the group generated by τ and negation acting on $E_a(K)$. Since $\tau^n = 1$, $(-1)^2 = 1$ and the two commute, Γ is a quotient of $\mathbb{Z}/n\mathbb{Z} \times \mathbb{Z}/2\mathbb{Z}$ and has order at most $2n$. Its orbits on $G \setminus \{O\}$ are the *signed-Frobenius orbits*; a factor base closed under Γ is Γ-*invariant*. The following standard fact is the foundation of everything that follows, and we state it with its hypothesis made explicit because the "exceptional short orbit" case is exactly the case in which it fails.

Lemma 2.1 (Free action)**.** *Suppose that $r \nmid \#E_a(\mathbb{F}_{2^d})$ for every proper divisor d of n. Then Γ acts freely on $G \setminus \{O\}$: if $\pm\tau^k(P) = P$ for some $P \in G \setminus \{O\}$ then $n \mid k$ and the sign is $+$. In particular every signed-Frobenius orbit in $G \setminus \{O\}$ has exactly $2n$ elements, and the x-coordinates $x, x^2, \ldots, x^{2^{n-1}}$ of its members are pairwise distinct. When n is prime the hypothesis reduces to $r > 4$.*

Proof. Let $d = \mathrm{ord}_r(\lambda)$. Since $\lambda^n \equiv 1$, $d \mid n$. If $d < n$ then τ^d fixes every point of G, and the fixed points of τ^d on $E_a(\bar{K})$ are exactly $E_a(\mathbb{F}_{2^d})$, so $G \subseteq E_a(\mathbb{F}_{2^d})$ and $r \mid \#E_a(\mathbb{F}_{2^d})$, contradicting the hypothesis. Hence $d = n$, which is odd. Now $\tau^k(P) = \pm P$ gives $\lambda^k \equiv \pm 1$, so $\lambda^{2k} \equiv 1$, so $n \mid 2k$, so $n \mid k$ since n is odd, and then $\lambda^k \equiv 1 \neq -1$ because r is odd. The claim about x-coordinates follows: $x^{2^i} = x^{2^j}$ with $0 \leq i < j < n$ would give $\tau^{j-i}(P) \in \{P, -P\}$. For n prime the only proper divisor is $d = 1$ and $\#E_a(\mathbb{F}_2) \in \{2, 4\}$. □

Remark 2.2 (Exceptional short orbits)**.** The points that do have short orbits are exactly those excluded by the lemma: O; the 2-torsion point $(0,1)$, fixed by both τ and -1; the points of $E_a(\mathbb{F}_2)$ (which for $a=0$ are $(1,0),(1,1)$, interchanged by negation and fixed by τ); and, for composite n, the points of the proper subfield curves $E_a(\mathbb{F}_{2^d})$. None of them lies in G when the hypothesis holds, but all of them do arise as x-coordinates in a decomposition pipeline (a summation-polynomial root $x \in \mathbb{F}_2$ is not rare at small n, and $x=0$ corresponds to a 2-torsion "solution"). A canonicalization routine therefore has to be *total* on $\mathbb{F}_{2^n}$ and must flag these words rather than mis-assign a phase to them. Section 4.3 makes this precise.

2.3 Normal bases and rotation

Fix a normal basis $\{\beta, \beta^2, \ldots, \beta^{2^{n-1}}\}$ of K over $\mathbb{F}_2$ and write $x = \sum_i w_i \beta^{2^i}$ with $w = (w_0, \ldots, w_{n-1}) \in \mathbb{F}_2^n$. Then x^2 has coordinate vector $\mathrm{rot}_1(w) = (w_{n-1}, w_0, \ldots, w_{n-2})$: squaring is a cyclic rotation, and the Frobenius orbit of x is the *necklace* of w. The element 1 has coordinate vector 1^n and 0 has 0^n; these are the only words fixed by every rotation, and for n prime they are the only words with any nontrivial rotational symmetry (a word with $\mathrm{rot}_j(w) = w$, $0 < j < n$, has period $\gcd(j,n) = 1$). This is the normal-basis form of Lemma 2.1: for prime n every word outside $\{0^n, 1^n\}$ has exactly n distinct rotations.

For the field sizes in this paper the normal basis is a Gaussian normal basis of type 2 (an optimal normal basis, ONB) when one exists ($n = 131$ for ECC2K-130, $n = 83$); the repository's packed arithmetic computes multiplication in a polynomial basis and converts, so that squaring, Frobenius powers and the canonicalization below cost only word rotations [2]. A permutation basis-change costs one table pass per direction and is charged in every pipeline count reported here.

2.4 Index calculus with a factor base of orbits

Let $F \subset G$ be a factor base and $P, Q \in G$ with $Q = [s]P$ to be found. Relation collection draws random $R = [\alpha]P + [\beta]Q$ and tries to decompose $R = \sum_{i=1}^{m} \pm P_i$ with $P_i \in F$ by Semaev's summation polynomial S_{m+1} [24], restricted to a subspace of x-coordinates [6, 17]. Each success is a linear relation among the unknowns $\{\log_P P_i\}$, and once the relation matrix has rank $|F|$ (after cofactor clearing) a solve modulo r yields the logarithms and, from a relation involving Q, the answer s.

If F is Γ-invariant it is a union of $|F|/2n$ orbits. Choose a representative $\hat{P}_c$ in each orbit c. Every $P_i \in F$ is $P_i = \epsilon_i \tau^{k_i}(\hat{P}_{c(i)})$ for a unique sign $\epsilon_i \in \{\pm 1\}$ and phase $k_i \in \mathbb{Z}/n\mathbb{Z}$, so

$$\log_P P_i = \epsilon_i \lambda^{k_i} \log_P \hat{P}_{c(i)},$$

and a decomposition becomes a relation among the $|F|/2n$ unknowns $\log_P \hat{P}_c$ with coefficients in the order-$2n$ subgroup $\langle \lambda, -1 \rangle \leq (\mathbb{Z}/r\mathbb{Z})^\times$. The number of columns falls by $2n$ while the relation yield per attempt is unchanged, so the number of relations needed, and hence the number of decomposition attempts, falls by $2n$ and the linear-algebra cost by roughly $(2n)^2$. This is the factor-base symmetry exploited by Gallant et al. [16], Wiener and Zuccherato [26] for Pollard rho and by Faugère et al. [10], Galbraith and Gebregiyorgis [13] for index calculus; we recall it only to fix the notation and to make clear that the paper does not claim it. What the paper is about is the *cost of realizing it*: finding the orbit representative and the phase of a decomposed point, storing representatives compactly, probing them, and carrying the phase through relation reconstruction and verification without losing it.

3 The signed-Frobenius representation

3.1 Definition

Fix a total order $\prec$ on $K = \mathbb{F}_{2^n}$; in the implementation it is the numeric order of the normal-basis coordinate vector read most significant bit first, but nothing in this section depends on the choice. Fix also a *sign chooser*: a function $\sigma : E_a(K) \setminus \{O, (0,1)\} \to \{0,1\}$ with $\sigma(-P) = 1 - \sigma(P)$.

Definition 3.1 (Signed-Frobenius canonical form)**.** Let $P = (x, y) \in G \setminus \{O\}$ under the hypothesis of Lemma 2.1. The *phase* of P is the unique $k = \kappa(P) \in \{0, \ldots, n-1\}$ such that x^{2^k} is the $\prec$-least element of $\{x^{2^i} : 0 \le i < n\}$. Let $P' = \tau^k(P)$ and $\epsilon = (-1)^{\sigma(P')}$. The *canonical representative* of P is $\hat{P} = \epsilon P'$, and the *signed-Frobenius representation* of P is the triple

$$\mathrm{can}(P) = (\hat{P},\ \epsilon,\ k), \qquad \text{so that} \qquad P = \epsilon\, \tau^{-k}(\hat{P}) = \epsilon\, \tau^{n-k}(\hat{P}).$$

Uniqueness of k is Lemma 2.1: the n conjugates of x are distinct, so exactly one of them is $\prec$-least. By construction $\hat{P}$ depends only on the orbit ΓP: every $P'' \in \Gamma P$ has the same set of conjugate x-coordinates, hence the same least one, hence the same P' up to sign, and the sign chooser fixes the sign. So can is a section of the orbit map, and the *orbit identifier* of P may be taken to be $x(\hat{P})$ alone: the canonical x-coordinate determines $\hat{P}$ because σ decides between the two points above it.

Proposition 3.2 (Logarithm transport)**.** *With* $\mathrm{can}(P) = (\hat{P}, \epsilon, k)$ *and any generator* P_0 *of* G*,*

$$\log_{P_0} P \;=\; \epsilon\, \lambda^{n-k} \log_{P_0} \hat{P} \;=\; \epsilon\, \lambda^{-k} \log_{P_0} \hat{P} \pmod{r}.$$

Proof. τ^{n-k} acts on G as $[\lambda^{n-k}]$, and $\lambda^n \equiv 1$. □

3.2 A Frobenius-invariant sign chooser

The sign chooser can be anything deterministic, since in Definition 3.1 it is evaluated after the rotation. There is, however, a chooser that commutes with τ, which lets an implementation decide the sign before, after, or independently of the rotation, and store it as one bit next to the canonical x.

Lemma 3.3 (Trace sign)**.** *Let* n *be odd and* $P = (x, y)$ *with* $x \neq 0$*. Define* $\sigma(P) = \mathrm{Tr}_{K/\mathbb{F}_2}(y/x)$*. Then* $\sigma(-P) = 1 - \sigma(P)$ *and* $\sigma(\tau P) = \sigma(P)$*.*

Proof. $-P = (x, x + y)$ and $\mathrm{Tr}((x+y)/x) = \mathrm{Tr}(1) + \mathrm{Tr}(y/x) = n \bmod 2 + \mathrm{Tr}(y/x) = 1 + \mathrm{Tr}(y/x)$. The trace is invariant under squaring, $\mathrm{Tr}(z^2) = \mathrm{Tr}(z)$, and $\tau P = (x^2, y^2)$ has $y^2/x^2 = (y/x)^2$. □

Writing $z = y/x$, the curve equation becomes $z^2 + z = x + a + x^{-2}$, so σ is the bit that the curve equation leaves free: it is the standard point-compression bit of binary curves, computed in a trace form. The cost is one field inversion (or one division) per point, which is why the pipelines of Section 6 compute it only for the points that are actually stored, not for every candidate root. The decomposition stage works on x-coordinates alone: a summation-polynomial root is an x, and the two points above it are $\pm P$, so the sign is not known until a y is recovered. Lemma 3.3 says the sign can then be attached to the representative without undoing the rotation.

3.3 Phase-aware relations

A decomposition of $R = [\alpha]P_0 + [\beta]Q$ into $R = \sum_{i=1}^{m} \epsilon'_i P_i$ with $P_i \in F$ is first obtained as a set of x-coordinates x_i (roots of the restricted summation polynomial) together with a sign pattern

ϵ_i' fixed by checking the point sum. The *phase-aware relation* is obtained by canonicalizing each x_i:

$$\mathrm{can}(P_i) = (\hat{P}_{c(i)}, \epsilon_i, k_i), \qquad \alpha + \beta s \;\equiv\; \sum_{i=1}^{m} \epsilon_i' \epsilon_i \lambda^{-k_i}\, \ell_{c(i)} \pmod{r},$$

where $\ell_c = \log_{P_0} \hat{P}_c$ are the unknowns and $s = \log_{P_0} Q$. Two points of the same orbit in one decomposition (possible, and at small n common) simply add their coefficients in the same column; a point whose column coefficient becomes 0 mod r contributes nothing.

Theorem 3.4 (Correctness of phase-aware reconstruction)**.** *Assume the hypothesis of Lemma 2.1. Let $\mathcal{R}$ be a set of phase-aware relations whose coefficient matrix over $\mathbb{Z}/r\mathbb{Z}$ has full column rank on the columns it touches, and let (ℓ_c) be the solution. Then for every $P \in F$ with $\mathrm{can}(P) = (\hat{P}_c, \epsilon, k)$, $\log_{P_0} P = \epsilon\lambda^{-k}\ell_c$, and s is the value obtained from any relation with $\beta \neq 0$. If moreover each relation was verified as a point identity before being admitted, then every recovered logarithm satisfies $[\log_{P_0} P]P_0 = P$.*

Proof. Each admitted relation is a true identity in G by construction, and Proposition 3.2 shows that substituting the true logarithms $\ell_c^* = \log_{P_0} \hat{P}_c$ satisfies it. Full column rank makes the solution unique, so $\ell_c = \ell_c^*$, and transport gives the remaining logarithms. Verification as a point identity is exactly the statement $[\log]P_0 = P$. □

The theorem is elementary, and that is the point: once the three pieces of $\mathrm{can}(P)$ are carried through, there is nothing left to prove about the linear algebra. The engineering content is in making can cheap and in not losing k between the decomposition stage (which sees only x), the table (which stores only $\hat{x}$) and the matrix (which needs λ^{-k}). Section 6 describes where each piece lives, and Section 7 reports complete runs in which every logarithm was checked.

Remark 3.5 (Composite n and the general case)**.** If n is composite or the hypothesis of Lemma 2.1 fails, the phase is only defined modulo the length $d \mid n$ of the x-orbit, and $\lambda^d \equiv 1$ need not hold on the points in question (they are outside G). The representation remains correct if κ returns the *least* k achieving the minimum and the pipeline rejects points outside G at verification time, which it must do anyway: a relation involving a point of the wrong order is not an identity in G and fails the point check. We implement the general rule (least shift) and test the prime case, which is the only one used.

4 Canonicalization by cyclic zero gaps

Computing $\kappa(P)$ is the inner loop of every phase-aware pipeline: each candidate x produced by the decomposition stage has to be canonicalized before it can be looked up, and in the probe-heavy regimes of Section 7 there are 10^6 to 10^7 candidates per target. In a normal basis the problem is exactly the classical *necklace* or *least rotation* problem on an n-bit cyclic word, and the baseline implementation does what the definition says: form all n rotations and keep the least. This section gives an exact shortcut, proves it, and specifies its behaviour on the exceptional words of Remark 2.2.

4.1 The lemma

For $w \in \{0,1\}^n$ let $\mathrm{rot}_j(w)$ be the left rotation by j (so that $\mathrm{rot}_j(w)$ begins with w_j), $\mathrm{val}(w) \in [0, 2^n)$ the integer with w_0 as most significant bit, and $\mathrm{lz}(w)$ the number of leading zeros. A *cyclic zero run* of w is a maximal set of cyclically consecutive positions holding 0; its *start* is the position after the preceding 1.

The following fact is standard for necklaces [12, 23]; we state it in the form the implementations use and prove it for completeness.

Lemma 4.1 (Zero gaps)**.** *Let $w \neq 0^n, 1^n$ and let Z be the length of a longest cyclic zero run of w. Then every j minimizing $\mathrm{val}(\mathrm{rot}_j(w))$ is the start of a cyclic zero run of length Z. Consequently*

$$\min_j \mathrm{val}(\mathrm{rot}_j(w)) = \min\{\mathrm{val}(\mathrm{rot}_j(w)) : \ j \text{ starts a zero run of length } Z\}.$$

Proof. If $\mathrm{lz}(u) = z < n$ then $2^{n-z-1} \leq \mathrm{val}(u) < 2^{n-z}$, because the first one bit of u sits at position z. Hence $\mathrm{lz}(u) > \mathrm{lz}(u')$ implies $\mathrm{val}(u) < \mathrm{val}(u')$. For any j, $\mathrm{lz}(\mathrm{rot}_j(w)) \leq Z$, with equality exactly when j starts a zero run of length Z; such a j exists because $w \neq 1^n$ has a zero run and $w \neq 0^n$ makes runs finite. A rotation not starting at such a j has strictly fewer leading zeros than one that does, hence a strictly larger value, so it is not a minimizer. □

Corollary 4.2 (Uniqueness for prime n)**.** *If n is prime and $w \neq 0^n, 1^n$, the minimizing j is unique.*

Proof. $\mathrm{val}(\mathrm{rot}_i(w)) = \mathrm{val}(\mathrm{rot}_j(w))$ with $i \neq j$ means $\mathrm{rot}_{j-i}(w) = w$, so w has period $\gcd(j - i, n) = 1$ and is constant. □

4.2 Two implementations

Lemma 4.1 leaves the implementation a choice of how to find the longest zero runs. The two code bases make different choices, both exact, and both are measured in Section 7.2.

Variant A: bit-parallel run growth (one machine word, $n \leq 63$). Let $z = \neg w$ (masked to n bits). The word $Z_1 = z$ has a one at every zero of w; $Z_k = Z_{k-1} \wedge \mathrm{rot}_{k-1}(z)$ has a one at position p exactly when bits $p, p-1, \ldots, p-k+1$ of w are all zero. Growing k until $Z_{k+1} = 0$ takes Z steps of one AND and one rotate, about $\log_2 n$ for a random word, and leaves in Z_Z the top position of every longest run. The candidates are then rotated and compared one by one; ties take the smallest shift. This is `FrobeniusCanon::least_rotation` in the sibling repository.[1] On 2×10^6 uniform 61-bit words 72.6% have one longest run, 17.9% two, so the candidate loop usually runs once.[2]

Variant B: set-bit scan (two limbs, $n = 83$). Algorithm 1 enumerates the set bits of w with count-trailing-zeros, computes each cyclic gap between consecutive ones, and keeps a running maximum. It rotates the word at every gap that equals or exceeds the running maximum, in scan order, and keeps the least rotation, breaking ties by the smaller shift; the wrap-around gap between the last and the first one bit is handled last. The two constant words are detected first and return phase 0, the convention Remark 2.2 requires. This is `canonical_gap` in the $n = 83$ index.[3] Rotating at running maxima rather than only at the final maxima avoids a second pass over the gaps and costs a few extra rotations: the expected number of records in a sequence of t gaps is $H_t \approx \ln t$, and with ties the measured figure is 4.78 rotations per word at $n = 83$.

Variant C: the exhaustive scan, vectorized. On AVX-512 the exhaustive $n - 1$ rotate-and-minimum chain on sixteen keys at once has no data-dependent branch. Alone it is slower than variant A (16.6 against 13.4 ns a key at $n = 53$), but inside the decomposition scan it is faster, 39–41 against 61 ns a summand, because variant A's two data-dependent loop exits (the run-length growth and the candidate loop) mispredict on most keys and stop the out-of-order core from overlapping consecutive independent keys.[4] A branch-free laned form of variant A (binary lifting with selects) recovers part of this on the portable path (Section 7.2).

[1] `crypto:src/cryptanalysis/koblitz_fast.rs, least_rotation.`

[2] `crypto:research/canon_laned_keys_20261008/README.md.`

[3] `experiments/koblitz-n83-index-design-20260929/packed_index_e1_canonical_20261009/native_packed_index.rs, canonical_gap.`

[4] `crypto:src/cryptanalysis/koblitz_fast.rs, canon_many; crypto:research/canon_laned_keys_20261008/README.md.`

Algorithm 1 Zero-gap canonicalization, variant B, of an n-bit normal-basis word.

Require: $w \in \{0,1\}^n$; n prime
Ensure: $(\hat{w}, k)$ with $\hat{w} = \mathrm{rot}_k(w) = \min_j \mathrm{rot}_j(w)$ and k least
1: **if** $w = 0^n$ or $w = 1^n$ **then return** $(w, 0)$ ▷ exceptional: $x \in \mathbb{F}_2$
2: **end if**
3: $(\hat{w}, k, G) \leftarrow (w, 0, 0)$ ▷ best word, best shift, best gap
4: $p_{\mathrm{first}} \leftarrow \mathrm{ctz}(w)$; $p \leftarrow p_{\mathrm{first}}$; clear bit p of a copy w' of w
5: **while** $w' \neq 0$ **do**
6: $p' \leftarrow \mathrm{ctz}(w')$; $g \leftarrow p' - p - 1$; CONSIDER(g, p'); $p \leftarrow p'$; clear bit p' of w'
7: **end while**
8: CONSIDER$(p_{\mathrm{first}} + n - p - 1, p_{\mathrm{first}})$ ▷ cyclic wrap-around gap
9: **return** $(\hat{w}, k)$
10: **procedure** CONSIDER(g, q) ▷ g zeros end just below the one bit at q
11: **if** $g = 0$ or $g < G$ **then return**
12: **end if**
13: $j \leftarrow (n - q) \bmod n$; $u \leftarrow \mathrm{rot}_j(w)$ ▷ one rotation of the whole word
14: **if** $g > G$ or $u < \hat{w}$ or $(u = \hat{w}$ and $j < k)$ **then** $(\hat{w}, k, G) \leftarrow (u, j, g)$
15: **end if**
16: **end procedure**

Proposition 4.3 (Cost)**.** *Let $L = \lceil n/64 \rceil$ and $c(w)$ the number of longest zero runs of w. Variant A performs $Z(w) \leq n$ AND-rotate steps and $c(w)$ rotate-and-compare steps; variant B one popcount, one ctz scan over the $t \leq n$ one bits, and one rotate-and-compare per running-maximum gap. The exhaustive scan performs $n - 1$ rotate-and-compare steps. For w uniform on $\{0,1\}^n$, $\mathbb{E}[Z(w)] = O(\log n)$, $\mathbb{E}[c(w)] = O(1)$ and the expected number of running maxima is $O(\log n)$. Variant A therefore performs $O(L \log n)$ word operations against $\Theta(nL)$ for the exhaustive scan. Variant B's ctz scan visits every one bit, $\mathbb{E}[t] = n/2$, so it performs $\Theta(n)$ scalar steps plus $O(L \log n)$ word operations for its rotations; it saves rotations, not steps. The number of whole-word rotations falls from $n - 1$ to $O(1)$ (variant A) or $O(\log n)$ (variant B).*

The measured constants are in Section 7.2: on 8×10^6 words at $n = 83$ variant B performed 4.78 full rotations per word against the exhaustive scan's 82, a 17.1× reduction in rotations, and returned the identical $(\hat{w}, k)$ on every word.

4.3 Correctness, including the exceptional words

Theorem 4.4. *For prime n, variants A and B return, for every $w \in \{0,1\}^n$, the same pair $(\hat{w}, k)$ as the exhaustive scan that evaluates all n rotations and keeps the first minimum.*

Proof. For $w \in \{0^n, 1^n\}$ all rotations are equal, the exhaustive scan keeps $k = 0$, and so do both variants. Otherwise Lemma 4.1 says the minimum is attained among the starts of the longest zero runs; variant A visits exactly those, and variant B visits a superset of them (every running-maximum gap, including all gaps equal to the final maximum, since a gap equal to the current best is not skipped). Corollary 4.2 says the minimum is attained once, so whichever candidates are compared, the least word is the same and its shift is unique. □

Variant B is checked against the exhaustive scan on every word of lengths $3, 5, 7, 11$ and on 10^5 fixed-seed 83-bit words; variant A is checked on every word of composite lengths $8, 12, 16, 20$ as well, where periodic words make several rotations tie and the smallest shift must win,[5] and the

[5] `crypto:src/cryptanalysis/koblitz_fast.rs, the_least_rotation_partitions_exactly_as_the_squaring_chain`.

end-to-end runs of Section 7.2 compare the complete set of canonical keys and shifts produced by the two arms on 8×10^6 words.[6]

Remark 4.5 (Why not a linear-time least-rotation algorithm)**.** Booth's and Duval's algorithms find the least rotation of a string in $O(n)$ character comparisons. On an 83- or 131-bit word held in two or three limbs they are not the right tool: their $O(n)$ is in *bit* comparisons with data-dependent branches, whereas a limb rotation plus compare is a handful of straight-line instructions, so the question is how many whole-word rotations are needed, not how many bits are touched. Lemma 4.1 reduces that number to the number of longest zero runs, and in the measured workload that is under five.

5 Phase-aware memory layouts

A Γ-invariant factor base has $2n$ times more points than columns. Every table a pipeline keeps, the factor base itself, the table of decomposed roots that a meet-in-the-middle probe looks up, and the pair-sum table that a two-summand descent probes, can be stored per orbit instead of per point, at the price of canonicalizing every query. This section records the layouts the implementations use, the bit budgets that make the phase free to store, and two observations about canonical keys that are easy to get wrong.

5.1 Representatives on disk

The factor base is stored as one x-coordinate per signed orbit. For the $n = 83$ instance of Section 7.2 the base of 30 977 592 usable points is 186 612 representatives, and $30\,977\,592/186\,612 = 166 = 2n$ exactly, which is Lemma 2.1 checked on the data: every orbit is full. Each representative is 16 bytes (the polynomial-basis x, little-endian), so the base is 2.99 MB instead of the 496 MB that the points would occupy at the same width.[7] Nothing else needs to be stored: the sign is recovered from the canonical x by the chooser, and the phase is recomputed, not stored, because it is a function of the query, not of the base.

5.2 The packed root record

The meet-in-the-middle decomposition at $n = 83$ indexes the roots of the summation polynomial S_3 over sampled pairs of base points and then probes the index with the roots obtained from the target. The index record is 18 bytes:

$$\underbrace{\hat{x}}_{\text{11 bytes: exact 83-bit canonical key}} \;\|\; \underbrace{\text{offset}}_{\text{42 bits}}\ \underbrace{\text{root}}_{\text{1 bit}}\ \underbrace{k}_{\text{7 bits}} \quad \text{(7-byte descriptor)},$$

where the offset addresses the satisfiable pair-state vector, the root bit says which of the two S_3 roots this is, and $k < 83$ is the canonical shift of Definition 3.1.[8] The descriptor has 56 bits and uses 50; the phase costs nothing beyond the bits that would otherwise pad the record. A probe canonicalizes the partner root to $(\hat{x}', k')$, looks up $\hat{x}'$, and recovers the relative Frobenius power as $(k + n - k') \bmod n$, which is the exponent of τ that carries the stored root onto the query. That is the only place the phase is consumed before relation reconstruction, and it is an addition modulo n.

Records are sorted into 2^{22} buckets by a 22-bit *mixed* prefix of the key (counting scatter, then a short sort inside each bucket), with a directory of 32-bit offsets. For 4×10^6 records the

[6] `experiments/koblitz-n83-index-design-20260929/packed_index_e1_canonical_20261009/README.md`.

[7] `experiments/koblitz-n83-index-design-20260929/adaptive_all_orientations_20261004/all83_ordinary_manifest.json`.

[8] `experiments/koblitz-n83-index-design-20260929/packed_index_e1_canonical_20261009/native_packed_index.rs, encode`.

table is $4\,000\,000 \times 18 + 2^{22} \times 4 + 4 = 88\,777\,220$ bytes, against 192.0 MiB for the open-addressed hash table it replaced, a 2.27× reduction in table bytes at the same record count (the first layout, a 20-bit raw prefix with 80 388 616 bytes, had been 2.50×).[9] Since $4 \times 10^6 < 2^{22}$ an offset needs 22 bits, and the ten spare bits of each directory word hold an 8-bit key fingerprint and a singleton flag; a fingerprint mismatch on a singleton bucket rejects the query before any record is loaded. On the frozen probe this cut full-key comparisons from 9 324 648 to 4 943 112, with 2 912 395 queries rejected by the tag alone.[10]

5.3 Canonical keys are skewed

A canonical key is a *minimum* over n rotations, so its leading bits are far from uniform. By Lemma 4.1 it begins with Z zeros where Z is the longest cyclic zero run of the word, and for a uniform n-bit word $\Pr[Z \geq z] \approx 1 - \exp(-n/2^{z+1})$, so at $n = 83$ the top five bits are zero with probability about 0.73 and the top six with probability about 0.48. A directory indexed by the raw leading bits of the key therefore collapses into a small fraction of its buckets. This was observed twice, independently, before it was understood: a folded pair table that bucketed on the key's high bits "piled into the first few buckets", invisible on misses and expensive on every hit, and the first packed index at $n = 83$ performed 58 251 024 full-key comparisons with a raw 20-bit prefix against 25 265 242 with a hashed one.[11] The rule that follows is simple and we state it as one: *never index a canonical key by its leading bits; hash it, or index by a mixed prefix.* The same skew is a feature for compression. A canonical key has $\lfloor \log_2 n \rfloor$ or more leading zeros with constant probability, so a variable-length encoding of the key saves a few bits per record; the implementations do not exploit this, and we record it as an open engineering item.

5.4 The folded pair table and the orbit tag

The two-summand descent asks, for a residual R, whether $R = P_i + P_j$ for base points P_i, P_j, by looking $x(R)$ up in a table of all pair sums. Since the base is Γ-invariant the set of pair sums is Γ-invariant too ($\tau(P + Q) = \tau P + \tau Q$ and $-(P + Q) = (-P) + (-Q)$), so the table needs one key per orbit of sums. Three tiers exist in the implementation: *full* (every pair, with both summands stored), *compact* (every pair, summands recovered by a scan) and *folded* (one key per orbit of sums, canonical key as above). The folded tier stores $2n$ times fewer entries and therefore affords a base $\sqrt{2n}$ times wider in the same memory, which divides the descent's expected $2r/|F|^2$ probes by $2n$; it pays with a canonicalization on every probe.

Two details make the folded tier usable rather than merely small.

Enumerating each orbit of sums once. Walking i over one representative of each signed orbit and j over the whole base reaches every orbit of ordered pairs (carry P_i to its representative by some $g \in \Gamma$; then (gP_i, gP_j) is enumerated), so no sum orbit is missed. But a sum orbit is reached twice, once from each summand's orbit, and the first implementation stored it twice, which made the fold worth n rather than $2n$ and made an "equal memory" comparison not equal. The row filter "row α keeps j only when $\mathrm{orb}(j) \geq \alpha$" stores each sum orbit once and keeps the covering property.[12]

Recovery in $O(n)$, not $O(|F|)$. A folded entry says only that the orbit of R contains a sum $P_{\mathrm{rep}(\alpha)} + P_j$. Recovering the actual summands of R, which the relation needs, would mean scanning the base. Storing the orbit index α of one summand in the high half of the entry's

[9] `experiments/koblitz-n83-index-design-20260929/packed_index_e1_prefix_20261007/README.md`.

[10] `experiments/koblitz-n83-index-design-20260929/packed_index_e1_tag_20261009/README.md`.

[11] `crypto:docs/ic/runs/koblitz-orbit-fold-20260913.json, a_bug_the_tag_exposed; experiments/koblitz-n83-index-design-20260929/packed_index_e1_hashed_prefix_20261008/README.md`.

[12] `crypto:docs/ic/runs/koblitz-orbit-fold-20260913.json, the_row_filter`.

"rest" word (the bits the bucket hash does not already determine) turns the scan into a walk over the $2n$ points of orbit α: $R = g^{-1}P_{\mathrm{rep}(\alpha)} + g^{-1}P_j$ with the first summand in orbit α, so one tries the $2n$ members of that orbit and subtracts. Measured at $n = 61$ the recovery fell from 8.66 ms to 0.037 ms per decomposition, from the majority of a decomposition to 1.4% of it, at no cost in table bytes.[13] This is the second place the phase is consumed: the orbit tag is the "which column" half of $\mathrm{can}(P)$, and the walk over the orbit is the "which phase" half, computed on demand.

The degenerate key. O is the sum of every $\pm$ pair, so the orbit of O, the single element O, is a key under which every orbit representative stores an entry. A lookup there returns every orbit at once. It is bounded (14.02 ms for the 88 816 pairs it legitimately returns at the measured width) once the two membership tests inside it are sorts rather than scans, and it is a reminder that Remark 2.2 applies to sums as well as to points.

6 Carrying the phase through a pipeline

This section fixes where each component of $\mathrm{can}(P) = (\hat{P}, \epsilon, k)$ is computed, stored and consumed in the three implementations the benchmarks are drawn from, so that the numbers in Section 7 can be read against a definite data flow. The three are (i) the Rust index-calculus pipeline with a folded pair table and the signed-Frobenius Pollard rho reference it is measured against, both in the sibling repository (`crypto`, degrees $n \leq 63$ in one machine word); (ii) the $n = 83$ packed root index of Section 5.2 (`cryptanalysis`, two-limb words); and (iii) the Sage pipeline for the $n = 19$ isogeny-class study, which is the one that produces verified logarithms with a Γ-invariant base and a Gröbner decomposition.

6.1 One convention per pipeline

Four canonical-representative conventions coexist in the code, and they agree on *which* points form an orbit but not on *which* member represents it:

1. *First met.* The factor-base builder visits $\pm\tau^k(P)$ for $k = 0, \ldots, n-1$ from each unassigned point and records (orbit, k, negated) on first sight. Cheap, order-dependent, and used only where the base is enumerated once.[14]

2. *Lexicographic minimum over the* $2n$ *members* of $\{\pm\tau^k([h]P)\}$ by the key $(x+1, y)$, after cofactor projection. Deterministic and basis-independent; costs $4\,n$ coordinate squarings per column to build. Used for the relation columns.[15]

3. *Least rotation in a normal basis* of the x word, with the sign chosen afterwards by comparing y with $y + x$ (in normal coordinates in the rho walk, in polynomial coordinates in the boundary harness). This is Definition 3.1 with $\prec$ the numeric order of the normal-basis word, and it is the convention every per-probe canonicalization uses.[16]

4. *Least numeric rotation in the permuted type-II ONB* of the ECC2K-130 client, where τ is a coordinate permutation rather than a rotation, and a different member of the orbit is chosen than by (3).[17]

[13] `crypto:docs/ic/runs/koblitz-orbit-fold-20260913.json, the_orbit_tag.`
[14] `crypto:src/cryptanalysis/koblitz_index_calculus.rs, orbit_maps_packed.`
[15] `crypto:src/cryptanalysis/koblitz_index_calculus.rs, build_projected_signed_orbit_map.`
[16] `crypto:src/cryptanalysis/koblitz_fast.rs, FrobeniusCanon; crypto:src/cryptanalysis/koblitz_strong_rho.rs, canonicalize.`
[17] `crypto:ecc2k130/CAIRN-WITNESS.md.`

The normal element in (3) is found by a bounded randomized search, so two independent builds name orbits by different, equally valid, incompatible keys. A folded table is therefore not portable across a basis change, and the GPU kernels import the host's basis-change tables rather than deriving their own.[18] We record this as a requirement for any reproducible artefact: *the normal element, or the basis-change tables, are part of the factor-base recipe and must be archived with it.*

6.2 The index-calculus data flow (`crypto`, $n \leq 63$)

1. *Factor base.* Points with x in a chosen set, closed under Γ; columns are signed orbits after cofactor projection (convention 2); a point with $[h]P = O$ gets no column.
2. *Pair table.* One of three tiers (Section 5.4); the folded tier keys by convention 3 on $x(P_i + P_j)$ and tags the row orbit.
3. *Collection.* For each random R, the $m = 3$ scan walks the base, forms $R - P_k$, canonicalizes its x (convention 3, blocked 1024 at a time) and probes the table. A hit yields a candidate pair; the orbit tag turns recovery into a walk over $2n$ points; the candidate relation is checked as a point identity before admission.
4. *Phase to coefficient.* Each admitted summand is mapped to its column by convention 2, giving $(c, k, \text{negated})$, and the coefficient $\pm\lambda^k$ is accumulated in column c (Proposition 3.2). Two summands in one column add.
5. *Linear algebra and recovery.* Sparse elimination modulo r; the target's own relation gives s; every recovered logarithm is verified as $[\log]P_0 = P$.

The phase appears twice (steps 3 and 4) and is never stored: the table stores the tag, the matrix stores λ^k.

6.3 The matched reference: signed-Frobenius Pollard rho

The reference against which every index-calculus number in this paper is read is Pollard rho on classes of Γ, the Wiener–Zuccherato and Gallant–Lambert–Vanstone walk with a $\sqrt{2n}$ saving [16, 26]. Its iteration function has to be class-invariant, and the implementation makes it so by the same canonicalization as the index-calculus side: least rotation of the x word in a normal basis, sign by comparing y with $y + x$ in normal coordinates, orbit multiplier $\pm\lambda^k$ from a table, with an exhaustive fallback over the $2n$ members when the word is constant.[19] So the two arms of every comparison pay the same canonicalization for the same automorphisms, which is the condition under which the comparison measures the index calculus and not the representation. Section 7.3 shows how much of the reference's own cost the canonicalization is, which is the reason this condition matters.

6.4 The $n = 83$ root index (`cryptanalysis`)

The decomposition is a meet-in-the-middle on roots of S_3: for sampled pairs of base points the two roots are canonicalized (convention 3, Algorithm 1) and stored with their shift (Section 5.2); the target's roots are canonicalized and probed; a hit gives the relative shift by subtraction modulo n, and the candidate relation is checked natively. The published runs stop at the frozen state cap with no relation, so this pipeline contributes stage measurements only (Section 7.2).

[18] `crypto:src/cryptanalysis/koblitz_fast.rs, FrobeniusCanon::new`.
[19] `crypto:src/cryptanalysis/koblitz_strong_rho.rs, canonicalize`.

6.5 The $n = 19$ Sage pipeline (`cryptanalysis`)

The isogeny-class study runs complete index calculus on $E_0/\mathbb{F}_{2^{19}}$ with a prime subgroup of order 130 873 and compares a plain subspace base, $x \in V = \langle 1, z, \ldots, z^9 \rangle$, with the Γ-closure of a smaller subspace, $F = \{\tau^j P : x(P) \in V'\}$ with $\dim V' \in \{6, 7\}$. Orbit representatives are $\min_e x^{2^e}$ in the integer encoding of the polynomial basis (a convention 2 choice, affordable at $n = 19$), and each base point is stored as $(P, c, e, \pm 1)$. The decomposition $R = P_1 \pm \tau^j P_2$ is a $2 \dim V'$-variable quadratic Boolean system for each shift j (squaring is $\mathbb{F}_2$-linear, so τ^j on V' is a linear change of the second block of variables); the shifts are tried in a random order and solved by PolyBoRi Gröbner bases. A hit returns $[(c_1, \epsilon_1, e_1), (c_2, \epsilon_2, (j + e_2) \bmod n)]$ and the row gets $\epsilon\lambda^e$ in column c, with λ the root of $\lambda^2 + \lambda + 2$ for which $[\lambda]G = (x_G^2, y_G^2)$.[20] The matrix is solved by a sparse left kernel modulo r, and each of the ten planted logarithms is checked against its scalar. This is the pipeline behind the only end-to-end, verified, phase-aware speedup in the paper (Section 7.5).

7 Benchmarks

7.1 How the numbers are read

Every measurement below comes from a committed artefact in one of the two repositories and is footnoted to it. We follow the measurement contract those repositories impose on themselves [1], because its distinctions are exactly the ones this paper needs to keep:

- *Unit.* Where a whole pipeline is priced, the unit is $S = \text{operations}/\sqrt{r}$ in batched group additions, read against a matched signed-Frobenius Pollard rho on the same instance (Section 6.3) and the generic floor $\sqrt{\pi/(2 \cdot 2n)}$. A ratio over a smaller slice of the pipeline is a *stage diagnostic* and is labelled as such.
- *Class.* A change is an *advance* if the ratio to the floor fell, *engineering* if S fell and the ratio to the floor did not, *relabelling* if a headline count fell while S rose, and *accounting* if only the unit changed. Nothing in this paper is an advance, and we say which of the other three each row is.
- *Isolation.* A CPU wall-clock ratio is promoted to a speedup only with a host-isolation receipt (exclusive cores, fixed frequency, A/A and quiet-host gates). Rows without one are *exploratory*; deterministic counts (operations, rotations, comparisons, bytes, retired instructions) do not need one.
- *Correctness.* A row without a verified answer is not a result. "Verified" means every recovered logarithm was checked as $[\log]P_0 = P$ against the planted scalar, in the run.

7.2 Canonicalization cost

Normal-basis rotation against the squaring chain. Table 1 prices one canonicalization at $n = 61$ four ways on one host: the obvious chain of $n - 1$ squarings with a running minimum (serial, and eight independent points in flight), the same chain with shift reduction for a sparse irreducible, and the normal-basis rotation of variant A after a byte-table basis change.[21] In the same unit the sibling ledger prices a canonicalization at $n = 41$ at 50.8 ns against 682.6 ns for a batched group addition, 0.074 of an addition, and across the rho ladder of Section 7.3 at 0.09–0.12 of an affine addition.[22] For contrast, the Python model of the ECC2K-130 relation

[20] `experiments/volcano-ic/tau.sage, tauRelation and tauDlp`.

[21] `crypto:examples/koblitz_fold_cost.rs; crypto:research/notes/ecc2k130/RESEARCH_KOBLITZ_INDEX_CALCULUS.md, "The table was 2n times larger"`.

[22] `crypto:research/notes/index-calculus/RESEARCH_IC_BOUNDARY_LEDGER.md, section 10.4 and section 19`.

Table 1: One canonicalization at $n = 61$, nanoseconds per key, one host, exploratory. Variant A is Section 4.2.

method	ns / key	relative
squaring chain, serial	846.8	9.9×
squaring chain, 8 lanes	733.7	8.6×
squaring with shift reduction	1299.3	15.2×
normal-basis rotation (variant A)	**85.5**	1×

Table 2: Exhaustive scan against zero-gap canonicalization (variant B) on the frozen $n = 83$ probe. Counts are exact; times are medians of three on an unisolated host.

measure	exhaustive	zero-gap
full rotations, index build	328 000 000	19 127 078
full rotations, target probe	656 000 000	38 261 354
exact key comparisons	9 324 648	9 324 648
root table, bytes	88 777 220	88 777 220
median index build	2.130 s	1.980 s
median conversion + canonicalization	2.216 s	1.557 s
median target stage	7.204 s	6.511 s

sweeps measured a squaring-chain canonicalization at 398.52 μs against 29.20 μs for a batched addition, 13.6 additions, which is the regime in which a folded table loses to a full one.[23]

Zero gaps against the exhaustive scan. Table 2 is the frozen $n = 83$ comparison of variant B with the exhaustive scan, on the same 2×10^6 sampled pair states, the same 4×10^6 root records and the same one target, six runs in the order baseline, zero-gap, zero-gap, baseline, baseline, zero-gap.[24] The rotation counts are deterministic: $656\,000\,000 = 8 \times 10^6 \times 82$ for the scan and 38 261 354, or 4.78 per word, for the shortcut, a 17.1× reduction. The canonical keys, shifts, first-witness samples, exact-comparison counts and every non-canonicalization operation count agreed between arms on all six runs. The timings are on an unisolated host and are stage diagnostics: the three adjacent target-stage pairs gave ratios 1.074, 1.156 and 1.118, and the canonicalization phase alone fell from 2.216 to 1.557 s (1.42×). An adapter that replays exactly this stage through the isolated benchmark service is committed; its receipt is not, so no CPU speedup is claimed for the shortcut.[25]

Branches, lanes and vectors. Under Callgrind the portable key path, an exhaustive loop over eight words, was 66.8% of all instructions of the frozen $n = 61$ cold pipeline, about 586 instructions per key over 74.4×10^6 keys. A branch-free laned form of variant A, measured on isolated cores against its parent commit with identical keys, relations and logarithms on all twelve cells, cut whole-pipeline time by 1.09–1.31× and retired instructions by 1.205× ($n = 53$) and 1.398× ($n = 61$); at $n = 41$ the gain was not established. Class: engineering.[26] An exploration of the key kernels alone on two isolated cores put a GFNI basis change at 0.82–0.94 ns a key against 2.9–4.3 for the byte tables, and a funnel rotation at 2.4–2.8 ns against 5.4–7.1 for the chained loop, with identical keys; this round is declared and has no end-to-end result yet.[27]

[23] `crypto:research/notes/ecc2k130/RESEARCH_ECC2K130_RELATION_SWEEPS.md`.

[24] `experiments/koblitz-n83-index-design-20260929/packed_index_e1_canonical_20261009/README.md`.

[25] `experiments/koblitz-n83-index-design-20260929/packed_index_e1_isolated_replay_20261010/README.md`.

[26] `crypto:research/canon_laned_keys_20261008/README.md`.

[27] `crypto:research/ic_tool_program/explorations/R06-key-gfni-20261006/README.md`.

Table 3: Retired instructions (Callgrind `Ir`) of the signed-Frobenius rho reference at $n = 53$, 1024 targets, all verified. R2 is the normal-basis canonicalization of Definition 3.1.

rung	change	Ir	IC / rho
R0	merged reference (software clmul)	371 102 176 689	0.240
R1	library field (hardware clmul)	281 027 207 019	0.317
R2	+ normal-coordinate canonicalization	68 926 289 783	1.294
R3	+ 32 lockstep lanes, batched inversion	46 384 789 571	**1.923**

Three limbs ($n = 131$). At the ECC2K-130 degree a word takes three 64-bit limbs (64+64+3 bits). A dependency-free crate implements five exact canonicalizers on that layout and agrees with the exhaustive scan on every word for $n \leq 20$ in several limb widths, on 10^7 uniform 131-bit words and on structured families (low and high weight, zero runs straddling limb boundaries, up to twelve tied longest runs, near-periodic words), with four planted mutants all caught.[28] In retired instructions per word (cachegrind, uniform words), the exhaustive scan costs 1850, variant B 1435 (5.43 rotations a word, but a ctz step for each of about 65 one bits), a branch-free refinement with a fixed 130 steps and mask selects 4510, and a binary-lifting form of variant A 514, the cheapest and not branch-free. On near-periodic words variant B rises to 2738, more than the exhaustive scan, while both branch-free arms retire the same count on every input; 72.4% of uniform words have a single longest run. No 131-bit normal-basis multiply exists in the code base, so the cost relative to a point operation was not measured, and the nanosecond times of that report come from an unisolated host. The branch-free form is therefore 2.4× the exhaustive scan and 8.8× the lifting form at this degree: being free of branches costs instructions here. Class: engineering, negative for the branch-free arm.

7.3 The canonicalization inside the reference

A signed-Frobenius rho walk that canonicalizes every step pays the cost of Section 7.2 once per step, and that cost turns out to dominate the walk. Table 3 is the instruction-counted ladder at $n = 53$, 1024 targets, in which each rung contains the previous ones.[29] In the merged reference (rung R0) the canonicalization routine was 50.82% of retired instructions. From R1 (the same walk on the library field) to R2, replacing it by the normal-basis least rotation with the sign chosen by comparing y, together with a λ^k table and a faster `mul_mod`, cut the whole process from 281.0 to 68.9 billion instructions, 4.08×, and still left the (then $\Theta(n)$) least-rotation scan at 51.5% of the final rung. The index-calculus arm it was measured against moved from 0.317 (R1) to 1.294 (R2) and 1.923 (R3) times the reference's cost without changing at all. The compact-orbit panel of Section 7.5 shows the same effect in CPU time: at $n = 53$ with 1024 targets the normal-basis walk used 4.76 CPU-seconds against 13.34 for the same batched engine with polynomial-basis canonicalization, 2.80×, while performing 5% *more* steps.[30]

Two consequences follow. First, a comparison of an orbit-folded index calculus against a rho that canonicalizes differently is a comparison of two canonicalizations, not of two algorithms; the "reversal" in Section 7.5 is exactly that. Second, Bailey et al.'s ECC2K-130 iteration $P \leftarrow P + \sigma^{j(P)}(P)$ with j a function of the normal-basis Hamming weight is class-invariant without canonicalizing each step, and normalizes only distinguished points [2]; the ledger prices a canonical step at 2.74–2.92 affine units against 1.38–1.41 for that step.[31] The reference used here is therefore not the cheapest signed-Frobenius rho that could be written, and every "vs rho" ratio below is conservative in the index calculus's favour by up to that factor.

[28] `experiments/canon-n131-20261014/README.md`.
[29] `crypto:research/notes/index-calculus/RESEARCH_STRONG_RHO_LADDER_20260929.md`.
[30] `crypto:research/notes/ecc2k130/compact_orbit_strong_rho_20260929/RESULT.md`.
[31] `crypto:research/notes/index-calculus/RESEARCH_IC_BOUNDARY_LEDGER.md, section 19`.

Table 4: Compact against folded pair table at equal memory (0.64 GiB each), $n = 61$, one host, exploratory. Hit rate is the fraction of residuals decomposed.

	compact	folded
base points	16 592	183 488
stored pairs	137 655 528	138 074 720
probes per target	1 183 373	9 676
ns per probe	108	331
hit rate	0.0043	0.9974
recovery per decomposition	1.18 ms	0.026 ms
seconds per decomposed target	0.500	0.00255

7.4 Orbit compression of tables

The factor-base pair table, operation-counted ($n = 41$). On the 39-bit Koblitz rung of the boundary ledger the Frobenius fold shrank the pair table from 6 249 999 to 152 439 entries (41.0×). Against the negation-folded base it replaced, the factor-base phase fell from 8.45 to 0.24 in S and relation collection rose from 41.9 to 45.0, paid in canonicalizations; net S went from 50.4 to 45.2 (from 58.8 for the unfolded base, whose factor-base phase was 16.9). On rungs with large cofactors, where the base was almost the whole cost, the same fold is worth more: $n = 29$ from 34 554 to 728, $n = 31$ from 2489 to 98.2. Class: engineering.[32]

The folded tier at equal memory ($n = 61$). Table 4 compares the compact and folded tiers of Section 5.4 at the same 0.64 GiB on one host, 48-bit subgroup, two-summand descent.[33] At equal bytes the folded tier holds a base 11.1× wider, which divides the expected probes per target by $|F|^2$ and the probes actually performed by 122×; the fold alone, on the code as it stood before the row filter and orbit tag, was worth 2.0×, and the rest of the 196× per decomposed target is the wider base made affordable. The prediction that a further $\sqrt{2}$ widening should be worth about 1.9× once recovery is 1.4% of a decomposition was tested on two runs and measured 1.93× and 2.04×, with every one of 21 779 and 22 619 targets decomposed. These are wall-clock per-decomposition figures on an unisolated host and a stage diagnostic; what they establish is that the orbit tag reconnected the lever the fold exists for, not an end-to-end speedup.

What a probe costs, in the shape the pipeline probes. The fold's probe penalty depends on the access pattern: 2.8× one target at a time, 2.0× in blocks of 1024 with prefetch, 1.51× inside the $m = 3$ scan. In the scan the folded table costs 74 ns a base point more than the compact one, and the canonicalization measured alone is 76 ns: the fold's cost in the descent is the canonicalization and nothing else, and blocking buys the folded table 1.83× because the $\sim$ 76 ns of dependent work before the address is known is what stops consecutive probes' memory round trips from overlapping.[34]

The tier choice, priced in operations. When the default tier was flipped to fold-first on the wall-clock evidence above and then re-priced over the whole cold pipeline in batched additions, the flip did not survive: at 12 688 points stored pairs fell 120.85× while total operations rose 1.2% (holdout targets: 1.016×). That is relabelling by definition. Re-asked in a separate sweep over four widths with every other count held identical, Table 5, folded is 2.7% dearer than full at 12 688 points and 12.0% cheaper at 18 544; each tier is cheapest somewhere: full below

[32] `crypto:research/notes/index-calculus/RESEARCH_IC_BOUNDARY_LEDGER.md, section 10.4.`
[33] `crypto:docs/ic/runs/koblitz-orbit-fold-20260913.json.`
[34] `crypto:docs/ic/runs/koblitz-probe-shape-20260913.json.`

Table 5: Total batched group additions of the cold pipeline at four factor-base widths, $n = 61$, 47-bit subgroup, 32 targets, all verified on every arm. Rho on the same instance: $S = 0.1774$; floor 0.1135.

points	orbits	full	compact	folded	cheapest
6 832	56	**876 342 501**	926 956 220	1 104 202 174	full
9 760	80	903 649 246	**880 008 741**	1 009 804 178	compact
12 688	104	941 440 872	**887 500 874**	966 559 218	compact
18 544	152	1 117 432 188	1 042 518 063	**982 920 004**	folded

about 8000 points, compact to about 16 000, folded above, with the full/folded crossover at $|F| \approx 13\,623$ against a pre-registered prediction of 13 600 from the measured per-probe deltas. The twelve cells cost 12.1–15.4× the matched rho's S and 18.9–24.1× the floor on this instance; the tier moves the constant only. Class: accounting for the sweep, relabelling for the flip.[35]

The packed root index ($n = 83$). The record of Section 5.2 holds the same 4×10^6 roots in 76.7 MiB against 192.0 MiB for the open-addressed hash table (2.50×), at a 1.41× slower probe in the first layout; the subsequent directory changes cut exact-key comparisons from 58 251 024 (raw prefix) to 25 265 242 (hashed prefix), 9 324 648 (22-bit buckets) and 4 943 112 (singleton tags), all deterministic, with unisolated timings that moved in both directions.[36]

Canonical pair tables as a cold job ($n = 19, 23$). Two preregistered, independently reviewed experiments in the research ledger measured the orbit-canonical pair table against the expanded one as complete fresh-process jobs.[37] At $n = 19$ (base of 152 signed points, four orbit columns) the canonical table has 293 keys against 11 097, an 8.1× smaller key/value payload, and a paired whole-process ratio of 0.751 $[0.655, 0.864]$ (a 1.33× speedup) over eight fixed targets; each hit retains a real pair witness, the shift and sign are inverted, and the recovered points are checked to sum to the residual. At $n = 23$ with 16 columns and 512 queries per table build the ratio was 0.647 $[0.638, 0.656]$, with 260 361 distinct pair sums held as 5 661 canonical entries; at $n = 19$ with 512 queries the two were at parity. The host was loaded; the Coordinator classified both as finite implementation measurements and declined knowledge promotion.

SAT encodings. A representative-plus-shift encoding of the $n = 53$ S_5 formula on 220 orbit columns has 22 887 clauses against 1 815 492 (79.3×), pending independent replay.[38] The same ledger found that an orbit-union encoding (one system per orbit instead of one per shift) is cost-neutral for the solver, ratios 1.000–1.467 and never below 1, and that a fixed-target orbit constraint is unsound: Frobenius carries one target's system to a conjugate target's system, not to itself.[39] Compression of the formula is therefore not the same as a faster solve, and the second claim is the one that has to be tested.

7.5 End to end, with verified logarithms

The Γ-invariant base with a Gröbner decomposition ($n = 19$). Table 6 is the complete-pipeline comparison of Section 6.5 on $E_0/\mathbb{F}_{2^{19}}$, ten planted scalars, CPU seconds per ECDLP,

[35] `crypto:docs/ic/runs/koblitz-tier-crossover-20260921.json`.

[36] `experiments/koblitz-n83-index-design-20260929/packed_index_e1_prefix_20261007/README.md`, and the sibling E-1 directories.

[37] `crypto-autoresearcher:research/n19_pair_transport_20260921/REPORT.md`; `crypto-autoresearcher:research/pair_reuse_20260922/REPORT.md`.

[38] `crypto:docs/ic/BOUNDARY_TARGETS.md`, factor_base row.

[39] `crypto-autoresearcher:knowledge/findings/KN-FIND-47da4e`; `crypto-autoresearcher:ledger/hypotheses/H-BINSTD-74b002.yaml`.

Table 6: Complete index calculus on $E_0/\mathbb{F}_{2^{19}}$, $r = 130\,873$, ten scalars each, all logarithms verified. Speedup is the paired mean of baseline/variant CPU with a 95% bootstrap interval.

factor base	decomposition	unknowns	relations	solver calls	CPU s / ECDLP	vs baseline
$x \in V$, dim 10	Gröbner	494	504	827	444	1×
Γ-orbits of $x \in V'$, dim 6	Gröbner, 19 shifts	29	39	5434	70	**6.85**× (5.70–8.02)
Γ-orbits of $x \in V'$, dim 7	Gröbner, 19 shifts	54	64	2664	72	6.26× (5.86–6.70)
normal-basis weight ≤ 3	CryptoMiniSat	35	45	48	668	0.68× (0.62–0.74)

speedup paired by scalar with a bootstrap interval; all 40 logarithms verified.[40] The orbit closure of a dimension-6 subspace needs 29 unknowns instead of 494 and 39 relations instead of 504; it pays with 19 shifts per decomposition attempt (5434 solver calls against 827), and still finishes in 70 s against 444. The textbook Γ-invariant base, normal-basis weight ≤ 3, is a different story: no 38-variable Gröbner solve finished, and with a SAT solver it is slower than the baseline. Three caveats are stated with the result: the unit is CPU time on a shared machine, the timing windows of the baseline were split by a hardware event, and the comparison is against a plain subspace base, not against rho. This is the only measurement in the paper in which a phase-aware base makes a complete, verified index calculus faster, and it is a toy.

The folded pipeline against rho ($n = 61$). The whole Rust pipeline with a 300 608-point, 2464-column base recovered 32 of 32 targets; the orbit tag and row filter cut collection 31.4×, the solve 64.9× and the descent 3.8× relative to the previous folded code, and peak memory from 4.68 to 2.40 GiB. Its own record charges it 330.7× the matched rho with setup included (86.7× before these changes, whose rho reference differed); the 12–15× of Table 5 belongs to the narrower 6.8k–18.5k-point sweep and is not this pipeline.[41]

Compact-orbit index calculus against the strong reference ($n = 37, 41, 53$). Table 7 is the isolation-gated cold panel: complete fresh-process CPU of a compact-orbit index calculus (base construction, full-rank collection, solve, verified recovery of one or 1024 targets) against the 32-walk normal-basis signed-Frobenius rho on the same public points, every cell past its preregistered A/A and quiet-host gates, 135 child processes, 46 170 target logarithms agreed by an independent replay on a second machine.[42] Every interval lies above parity. An earlier, separate experiment on different public points (2026-09-29, not isolation-gated) measured both references in one run: at its three batch cells the index calculus read 0.78, 0.39 and 0.34 of the *previous* rho reference (PCLMUL, Itoh–Tsujii), an apparent index-calculus win, and 2.08, 2.99 and 3.09 of the normal-basis reference. Those six numbers come from that experiment and are not cells of Table 7, which reads 3.03, 1.31 and 1.61 at the same nominal cells. The win was the reference's canonicalization, which is the point of Section 7.3.[43]

Online against cold: one target ($n = 41$–83). Single-target comparisons read differently depending on whether the index calculus's setup (base, relations, logarithm table) is charged. The boundary ledger's isolated single-target round separates the two readings on 64 public points at each of six sizes, $r = 2^{36.6}$–$2^{47.2}$, both arms on the same point.[44] With setup excluded the index calculus's online interval is 8.8–17.5× faster than single-target rho, every interval above 1 (4.3–9.4× if rho is priced at the canonical step). With setup included it is 2.0–31.6× slower at every size, and the setup pays for itself only after 7–37 targets. The same setup

[40] `experiments/volcano-ic/README.md, "E0 with a tau-invariant factor base".`

[41] `crypto:research/notes/ecc2k130/RESEARCH_KOBLITZ_INDEX_CALCULUS.md.`

[42] `crypto:docs/ic/BOUNDARY_TARGETS.md, "Complete cold compact-orbit panel"; crypto:research/notes/ecc2k130/disjoint_cold_v2_outcome_20261001/RESULT.md.`

[43] `crypto:research/notes/ecc2k130/compact_orbit_strong_rho_20260929/RESULT.md.`

[44] `crypto:research/notes/index-calculus/RESEARCH_IC_BOUNDARY_LEDGER.md, section 23.`

Table 7: Complete cold CPU, compact-orbit index calculus over signed-Frobenius rho, paired median with 95% interval, isolation-gated. K is the number of orbit columns.

cell	policy	IC / rho	reduction to parity
$n = 37$, 1 target, $K = 7$	unfiltered	2.810 [2.718, 2.874]	64.4%
$n = 37$, 1024 targets, $K = 42$	unfiltered	3.031 [2.998, 3.047]	67.0%
$n = 41$, 1 target, $K = 85$	unfiltered	13.378 [9.367, 18.491]	92.5%
$n = 41$, 1024 targets, $K = 255$	blocked prefilter	1.314 [1.288, 1.359]	23.9%
$n = 53$, 1 target, $K = 220$	unfiltered	22.675 [17.904, 31.595]	95.6%
$n = 53$, 1024 targets, $K = 440$	blocked prefilter	1.605 [1.566, 1.666]	37.7%

used as precomputation for Bernstein and Lange's generic walk would, by the ledger's model, reach an online cost 2.0–8.0× below the index calculus's, so the online advantage belongs to the precomputation and not to the method. A separate isolated panel at $n = 61$, $a = 0$ (64 unseen points per host, two hosts) reads 12.9× and 13.9× online (95% interval 9.8–19.1 on the pinned host) with the setup excluded and reported as a cold ratio of 83.4.[45] The earlier $n = 83$, $a = 1$ observation (three runs on one frozen point, median online ratio 8.2, shared host, no isolation receipt) is of the same online kind, and the ledger now keeps it as history.[46] None of these is an advance under Section 7.1: each reads a method with precomputation against a reference without it.

7.6 What survives complete relation collection

Table 8 collects the paper's improvements with what each one moved and its class. The pattern is uniform. Representation changes move constants, sometimes large ones, inside a method that remains slower than the matched rho at every degree measured once its setup is charged (1.3–331× across the rungs priced) and is faster online only by spending precomputation that a generic walk could spend better (Section 7.5); the same representation change inside the reference moves its constant by a comparable factor; and the headline count that a fold divides by $2n$ (entries, keys, clauses, unknowns) is never the count the end-to-end cost is proportional to.

8 Related work and what is not new here

Automorphisms in Pollard rho. Walking on classes of an automorphism group of order A saves $\sqrt{A}$ in the expected number of steps: negation [26], Frobenius on Koblitz curves [15, 16, 26], and the equivalence-class treatment of Duursma et al. [7]. The ECC2K-130 campaign [2, 3, 5] uses the $\langle \sigma, -1 \rangle$ classes of order 262 with an iteration $P \leftarrow P + \sigma^{j(P)}(P)$ whose branch j depends on the normal-basis Hamming weight, and normalizes a distinguished point "by taking the lexicographically smallest value among $x_R, x_R^2, \ldots, x_R^{2^{130}}$ in normal-basis representation" [2]. That is the canonical form of Definition 3.1 with $\prec$ the numeric order of the normal word; the form is not new, and neither is the fact behind the shortcut used to compute it (see *Least rotation* below). What this paper adds is the shortcut as an implementation with proofs of exactness (Section 4), the measurement of what it costs relative to the group operation it sits next to (Section 7.2), and the observation that in a walk that canonicalizes every step it is the dominant cost (Section 7.3).

Frobenius-invariant factor bases. Galbraith et al. [14] answer the question of how the Frobenius endomorphism can speed up index calculus on subfield curves: with a factor base

[45] `crypto:docs/ic/BOUNDARY_TARGETS.md, vs_rho row.`
[46] `crypto:experiments/koblitz-single-target-n83-20261006/.`

Table 8: Each improvement, what it changed, and its class under the contract of Section 7.1.

improvement	what moved	verified	class
normal-basis rotation vs squaring chain	8.6–15× per key; rho process 4.08× fewer instructions	keys identical; 1024/1024	engineering
zero-gap shortcut (variant B)	17.1× fewer rotations; canonicalization phase 1.42× (unisolated)	keys and shifts identical	engineering (stage)
laned variant A	1.09–1.31× pipeline (isolated)	identical logs	engineering
packed 18-byte record	2.27× fewer table bytes (2.50× first layout); 11.8× fewer comparisons across successive layouts on one frozen probe	controls identical	engineering (stage)
folded pair table	$2n$ fewer entries; +1.2% operations at 12688 pts; cheapest above $\approx 16\,000$ pts	32/32	relabelling / accounting
orbit tag	recovery $O(\|F\|) \to O(n)$; widening lever restored (1.9× predicted, 1.93–2.04 measured)	all targets decomposed	engineering
canonical pair table as a cold job	0.75 ($n = 19$) and 0.65 ($n = 23$) whole-job time, two separate experiments	witnesses verified	engineering
Γ-orbit Gröbner base ($n = 19$)	6.85× CPU per complete ECDLP vs plain base	40/40	engineering
representative+shift SAT encoding	79.3× fewer clauses; solve time unchanged	pending replay	relabelling

closed under π a relation can be "rewrit[ten] in terms of a smaller factor base consisting of representatives for each Frobenius orbit", which "allows to reduce the number of linearly independent relations we need to collect by a factor n" and reduces the linear-algebra cost by n^2; they construct such bases from Galois-invariant subspaces via linearised polynomials and from the Couveignes–Lercier isogeny construction, show how Frobenius-shifted bases give the full $m!$ symmetry-breaking saving without an m^2 linear-algebra penalty (their Theorem 3.2), and conclude that these improvements "do not lead to index calculus algorithms faster than Pollard ρ on instances used in practice". The idea descends from Galois-invariant factor bases in finite fields [20]. Everything in Section 2.4 is theirs. They define the orbit representative "in an ad-hoc manner" and do not price it; this holds in both the preproceedings and the peer-reviewed version of their paper, whose texts agree on every statement cited here; this paper is about that price and about the layouts that make the rewritten relations cheap to produce and verify. The sibling repository's structural finding that a Galois-invariant *subspace* base has a quotient gain of at most n and is a net loss of 57–96 bits at $n = 131$ and 73–122 bits at $n = 163$ for $m \leq 5$, because the π-stable subspaces of $\mathbb{F}_{2^n}$ have dimensions $\{0, 1, n-1, n\}$ when $\mathrm{ord}_n(2) = n - 1$, is the exact-spectrum counterpart of their Lemma 4.1 and is why the end-to-end evidence here stops at $n = 83$.[47]

Symmetries in the decomposition. Galbraith and Gebregiyorgis [13] reduce the factor base by torsion orbits (a factor 1/4 and then 1/8 on binary Edwards curves, with the matrix cost falling by the square), break symmetry for a net $(m-1)!$ and conclude that rho "is still much faster" at reasonable sizes; Faugère et al. [10, 11] use the symmetric group and small-order torsion to lower the degrees of the summation polynomials. The 2-torsion symmetrisation measured in the sibling repository (the `sym` arms of the sibling repository's symmetrised end-to-end study) is in that line and is orthogonal to the Frobenius phase: it lowers columns and degrees, never the trial count.

[47] `crypto-autoresearcher:knowledge/findings/KN-FIND-b9a41d`.

Index calculus on binary curves. The summation-polynomial method is Semaev [24], made subexponential in a range of extension degrees by Gaudry [17] and Diem [6]; the binary-field case with Gröbner bases is Faugère et al. [9], Petit and Quisquater [22], with the variable choices of Huang et al. [18], Shantz and Teske [25] and the linearised-polynomial view of Huang et al. [19]. None of these papers reports the cost of orbit canonicalization or a phase-carrying data layout, which is the gap this paper fills; all of them agree with the verdict of Section 7.5 that the method does not reach rho at cryptographic sizes.

Least rotation. Booth's and Duval's algorithms compute the least rotation of a string in linear time [4, 8]. Lemma 4.1 is not new as combinatorics: that a necklace begins with a longest run of its least letter is standard in the necklace-generation literature. Ruskey and Sawada use its density consequence to bound where the first nonzero letter of a fixed-density necklace can sit [23, §3.2], and Gabrić and Sawada make "a^ℓ is a prefix of w", with ℓ the length of a largest run of a's, the first clause of their definition of a quasinecklace [12, Definition 3]. What this paper adds is not the fact but its use as an exact candidate filter for word-sized normal-basis canonicalization: two branch-light implementations, a proof that both are bit-identical to the exhaustive scan including the exceptional words, and a measurement of how many whole-word rotations it leaves (4.78 per 83-bit word).

Measurement discipline. The unit, the classes and the isolation gate of Section 7.1 are taken from the companion measurement paper [1], which this paper should be read alongside: it supplies the matched rho and the floors; this paper supplies the representation underneath both.

9 Limitations and claims not made

The numbers do not compose. The results of Section 7 come from different experiments at different degrees with different pipelines, and they may not be multiplied. In particular two figures that circulated internally as "speedups", a 131× and an 18.9×, are not: the first is the ratio S/S_ρ of a meet-in-the-middle arm at $n = 23$ (it is 131 times *slower* than rho) and also the entry-count quotient of a quotiented ECC2K-130 table; the second is the compact tier's cost relative to the generic floor at $n = 61$ in the single-width run that preceded Table 5 (18.98–22.49 in the table itself).[48] Neither is a speedup of anything, and neither appears in this paper as one. Likewise the 6.85× of Table 6 is a CPU-time ratio between two factor bases at $n = 19$, not a ratio to rho, and should not be confused with the unrelated 6.85× that a Python Frobenius micro-benchmark happens to report at $n = 131$.[49]

No end-to-end advance. At every degree where a complete, verified pipeline was priced against the matched signed-Frobenius rho with its setup charged, the index calculus is slower: 12.1–15.4× in operations over the $n = 61$ tier sweep and 330.7× for the 300 608-point $n = 61$ pipeline, 1.3–23× in isolated CPU at $n = 37$–53, and 300× at the first, unfolded 39-bit rung of the boundary ledger (30–42× after folding). The isolated single-target rows in which it is faster online (8.8–17.5× at $r \le 2^{47}$; 12.9–13.9× at $n = 61$) exclude a setup that costs 62–292 online intervals and that a generic walk could use as precomputation to better effect (Section 7.5); the unisolated $n = 83$ observation (median 8.2×) is of the same kind and is not a claim. The paper

[48] 131: crypto:research/koblitz_symmetrised_e2e_20260927/RESULTS.md and crypto:research/isogeny_fibers_20261010/round4/16_1_m_crossover/RESULTS.md ($S=143$ against $S_\rho =1.09$); the entry-count quotient in crypto:research/notes/ecc2k130/RESEARCH_ECC2K130_RELATION_SWEEPS.md. 18.9: crypto:research/notes/ecc2k130/RESEARCH_KOBLITZ_INDEX_CALCULUS.md, the tier-flip table.

[49] experiments/sage-ic-campaign/onb-frobenius-20260924/RESULT.md.

claims no attack, no asymptotic statement and nothing about ECC2K-130 or any deployed curve.

The canonical form and the zero-gap fact are prior art. Section 8: the least Frobenius conjugate in normal-basis representation is the normalization of Bailey et al. [2], and Lemma 4.1 is a standard property of necklaces [12, 23]. The contributions are the implementations of the shortcut and their exactness proofs, the layouts and the measurements.

Isolation. Of the timing rows, only the laned-key result, the cold panel of Table 7 and the key-kernel exploration carry an isolation receipt. The zero-gap timings, the equal-memory fold, the probe shapes, the $n = 19$ Gröbner comparison and the cold-job pair tables are exploratory; their deterministic counts (rotations, comparisons, entries, bytes, instructions) are not.

Scope of the proofs. Lemma 2.1 and Theorem 4.4 assume n prime, which every measured instance satisfies; the composite case is implemented (least shift on ties) and tested on small composite n but not measured. The sign chooser of Lemma 3.3 is proved, not deployed: the implementations compare y with $y + x$ after the rotation, which is equivalent and cheaper when y is already in hand.

Naming is not portable. A normal element found by randomized search names orbits differently from another; a folded table, a root index or a distinguished-point log is readable only with the basis it was written in (Section 6.1). We treat this as an artefact requirement rather than a defect, but it is a real one.

What the shortcut does not fix. The zero-gap scan removes rotations, not branches. On a wide out-of-order core the data-dependent exits of variants A and B mispredict on most keys, and the exhaustive branch-free vector scan wins inside the decomposition loop at $n = 53$ (Section 4.2). Which variant is right depends on the loop it is embedded in. At $n = 131$ the branch-free three-limb arm retires 2.4× the exhaustive scan's instructions and the lifting form of variant A is cheapest (Section 7.2), but that is an instruction count on an unisolated host, not a timing inside a decomposition loop, and the right answer at $n = 83$ and $n = 131$ inside such a loop is open.

10 Conclusion

The $2n$ that a Frobenius-invariant factor base saves is a fact about counts; collecting it is a question of representation. We have given the representation, proved the three things an implementer needs to know about it (orbits are full for prime n, the sign can be chosen Frobenius-invariantly, and the least rotation of a necklace begins at a longest zero run), measured what it costs where it is paid, and shown with verified logarithms which of its consequences survive a complete pipeline. The honest summary is that the representation is cheap enough to be the right default on both sides of every comparison, that on the index-calculus side it buys memory and constants, and that on the Pollard rho side it buys a factor four in the reference, which is why a fair comparison needs it on both sides. The next measurements are fixed by the gaps in Section 9: an isolated replay of the zero-gap stage at $n = 83$, an isolated one-target comparison at $n = 83$ with the setup reported beside the online interval, a Bernstein–Lange reference arm for every online comparison, and a timing of the three-limb canonicalizers inside a decomposition loop at $n = 131$, where the branch-free form already costs more instructions than the exhaustive scan.

References

[1] Anonymous. The constant in front of $\sqrt{n}$: Measured Pollard rho, ECC2K-130 arithmetic, and index calculus over prime fields. Companion manuscript, `papers/ecdlp-constant/ecdlp.tex` in this repository, 2026.

[2] Daniel V. Bailey, Lejla Batina, Daniel J. Bernstein, Peter Birkner, Joppe W. Bos, Hsieh-Chung Chen, Chen-Mou Cheng, Gauthier van Damme, Giacomo de Meulenaer, Luis Julian Dominguez Perez, Junfeng Fan, Tim Güneysu, Frank Gurkaynak, Thorsten Kleinjung, Tanja Lange, Nele Mentens, Ruben Niederhagen, Christof Paar, Francesco Regazzoni, Peter Schwabe, Leif Uhsadel, Anthony Van Herrewege, and Bo-Yin Yang. Breaking ECC2K-130. Cryptology ePrint Archive, Paper 2009/541, 2009. URL https://eprint.iacr.org/2009/541.

[3] Daniel J. Bernstein, Hsieh-Chung Chen, Chen-Mou Cheng, Tanja Lange, Ruben Niederhagen, Peter Schwabe, and Bo-Yin Yang. ECC2K-130 on NVIDIA GPUs. In *Progress in Cryptology – INDOCRYPT 2010*, volume 6498 of *LNCS*, pages 328–346. Springer, 2010. doi: 10.1007/978-3-642-17401-8_23.

[4] Kellogg S. Booth. Lexicographically least circular substrings. *Information Processing Letters*, 10(4–5):240–242, 1980.

[5] Joppe W. Bos, Thorsten Kleinjung, Ruben Niederhagen, and Peter Schwabe. ECC2K-130 on Cell CPUs. In *Progress in Cryptology – AFRICACRYPT 2010*, volume 6055 of *LNCS*, pages 225–242. Springer, 2010. doi: 10.1007/978-3-642-12678-9_14.

[6] Claus Diem. On the discrete logarithm problem in elliptic curves. *Compositio Mathematica*, 147(1):75–104, 2011. doi: 10.1112/S0010437X10005075.

[7] Iwan Duursma, Pierrick Gaudry, and François Morain. Speeding up the discrete log computation on curves with automorphisms. In *Advances in Cryptology – ASIACRYPT 1999*, volume 1716 of *LNCS*, pages 103–121. Springer, 1999. doi: 10.1007/978-3-540-48000-6_10.

[8] Jean-Pierre Duval. Factorizing words over an ordered alphabet. *Journal of Algorithms*, 4 (4):363–381, 1983.

[9] Jean-Charles Faugère, Ludovic Perret, Christophe Petit, and Guénaël Renault. Improving the complexity of index calculus algorithms in elliptic curves over binary fields. In *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 27–44. Springer, 2012. doi: 10.1007/978-3-642-29011-4_4.

[10] Jean-Charles Faugère, Pierrick Gaudry, Louise Huot, and Guénaël Renault. Using symmetries in the index calculus for elliptic curves discrete logarithm. *Journal of Cryptology*, 27 (4):595–635, 2014. doi: 10.1007/s00145-013-9158-5.

[11] Jean-Charles Faugère, Louise Huot, Antoine Joux, Guénaël Renault, and Vanessa Vitse. Symmetrized summation polynomials: Using small order torsion points to speed up elliptic curve index calculus. In *Advances in Cryptology – EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 40–57. Springer, 2014. doi: 10.1007/978-3-642-55220-5_3.

[12] Daniel Gabrić and Joe Sawada. Necklaces and Lyndon words in colexicographic order. arXiv:2607.05324, 2026.

[13] Steven D. Galbraith and Shishay W. Gebregiyorgis. Summation polynomial algorithms for elliptic curves in characteristic two. In *Progress in Cryptology – INDOCRYPT 2014*, volume 8885 of *LNCS*, pages 409–427. Springer, 2014. doi: 10.1007/978-3-319-13039-2_24. Cryptology ePrint Archive, Paper 2014/806.

[14] Steven D. Galbraith, Robert Granger, Simon-Philipp Merz, and Christophe Petit. On index calculus algorithms for subfield curves. In *Selected Areas in Cryptography (SAC 2020)*, volume 12804 of *LNCS*, pages 115–138. Springer, 2021. doi: 10.1007/978-3-030-81652-0_5. Cryptology ePrint Archive, Paper 2020/1315.

[15] Robert P. Gallant, Robert J. Lambert, and Scott A. Vanstone. Improving the parallelized Pollard lambda search on anomalous binary curves. *Mathematics of Computation*, 69(232): 1699–1705, 2000. doi: 10.1090/S0025-5718-99-01119-9.

[16] Robert P. Gallant, Robert J. Lambert, and Scott A. Vanstone. Faster point multiplication on elliptic curves with efficient endomorphisms. In *Advances in Cryptology – CRYPTO 2001*, volume 2139 of *LNCS*, pages 190–200. Springer, 2001. doi: 10.1007/3-540-44647-8_11.

[17] Pierrick Gaudry. Index calculus for abelian varieties of small dimension and the elliptic curve discrete logarithm problem. *Journal of Symbolic Computation*, 44(12):1690–1702, 2009. doi: 10.1016/j.jsc.2008.08.005.

[18] Yun-Ju Huang, Christophe Petit, Naoyuki Shinohara, and Tsuyoshi Takagi. Improvement of Faugère et al.'s method to solve ECDLP. In *Advances in Information and Computer Security (IWSEC 2013)*, volume 8231 of *LNCS*, pages 115–132. Springer, 2013. doi: 10.1007/978-3-642-41383-4_8.

[19] Yun-Ju Huang, Christophe Petit, Naoyuki Shinohara, and Tsuyoshi Takagi. On generalized first fall degree assumptions. Cryptology ePrint Archive, Paper 2015/358, 2015. URL https://eprint.iacr.org/2015/358.

[20] Antoine Joux and Reynald Lercier. The function field sieve in the medium prime case. In *Advances in Cryptology – EUROCRYPT 2006*, volume 4004 of *LNCS*, pages 254–270. Springer, 2006. doi: 10.1007/11761679_16.

[21] Neal Koblitz. CM-curves with good cryptographic properties. In *Advances in Cryptology – CRYPTO '91*, volume 576 of *LNCS*, pages 279–287. Springer, 1992. doi: 10.1007/3-540-46766-1_22.

[22] Christophe Petit and Jean-Jacques Quisquater. On polynomial systems arising from a Weil descent. In *Advances in Cryptology – ASIACRYPT 2012*, volume 7658 of *LNCS*, pages 451–466. Springer, 2012. doi: 10.1007/978-3-642-34961-4_28.

[23] Frank Ruskey and Joe Sawada. An efficient algorithm for generating necklaces with fixed density. *SIAM Journal on Computing*, 29(2):671–684, 1999.

[24] Igor Semaev. Summation polynomials and the discrete logarithm problem on elliptic curves. Cryptology ePrint Archive, Paper 2004/031, 2004. URL https://eprint.iacr.org/2004/031.

[25] Michael Shantz and Edlyn Teske. Solving the elliptic curve discrete logarithm problem using Semaev polynomials, Weil descent and Gröbner basis methods – an experimental study. In *Number Theory and Cryptography*, volume 8260 of *LNCS*, pages 94–107. Springer, 2013. doi: 10.1007/978-3-642-42001-6_7.

[26] Michael J. Wiener and Robert J. Zuccherato. Faster attacks on elliptic curve cryptosystems. In *Selected Areas in Cryptography (SAC 1998)*, volume 1556 of *LNCS*, pages 190–200. Springer, 1999. doi: 10.1007/3-540-48892-8_15.

A Reproduction

Every number in the paper is footnoted to a file in one of three repositories: this library (`cryptanalysis`, paths without a prefix), the sibling Rust library (`crypto`, prefix `crypto:`) and the research ledger (prefix `crypto-autoresearcher:`). The artefacts that reproduce each section are:

Section 4, variant A
`crypto:src/cryptanalysis/koblitz_fast.rs` (`FrobeniusCanon`), test `the_least_rotation_partitions_exactly_as_the_squaring_chain`; cost: `crypto:examples/koblitz_fold_cost.rs` with argument `61`.

Section 4, variant B
`experiments/koblitz-n83-index-design-20260929/packed_index_e1_canonical_20261009/`, `python3 run.py` in a sibling copy (the committed receipts have fixed names); isolated replay adapter in `experiments/koblitz-n83-index-design-20260929/packed_index_e1_isolated_replay_20261010/`.

Section 5
the E-1 directories `packed_index_e1_prefix_20261007`, `packed_index_e1_hashed_prefix_20261008`, `packed_index_e1_bucket_20261008`, `packed_index_e1_linear_20261009`, `packed_index_e1_tag_20261009` under the same parent; `crypto:docs/ic/runs/koblitz-orbit-fold-20260913.json`.

Section 7.3
`crypto:research/notes/index-calculus/RESEARCH_STRONG_RHO_LADDER_20260929.md`; `crypto:research/notes/ecc2k130/compact_orbit_strong_rho_20260929/`.

Section 7.4
`crypto:docs/ic/runs/koblitz-tier-crossover-20260921.json` and, under `crypto:docs/ic/params/`, the files `k0n61-width<W>-<tier>.json` for $W \in \{6000, 9000, 12000, 18000\}$ and tier `full` or `folded`; `crypto:docs/ic/runs/koblitz-probe-shape-20260913.json`; `crypto:research/canon_laned_keys_20261008/`; `crypto-autoresearcher:research/n19_pair_transport_20260921/` and `crypto-autoresearcher:research/pair_reuse_20260922/`.

Section 7.5
`experiments/volcano-ic/` (`sage run.sage taucmp <shard> <shards>`, `sage -python analyze.py`); the archived base `experiments/fb-archive/sweeps/volcano-ic.json.gz`; `crypto:research/notes/ecc2k130/disjoint_cold_v2_outcome_20261001/` (frozen source `38c17307`, input freeze `da958a3f...`, manifest `350779eb...`); `crypto:experiments/koblitz-single-target-n83-20261006/`.

The factor bases are archived under `experiments/fb-archive/`; CI fails when a committed result cites a base digest that is not archived. The normal element (or the basis-change tables) used to name orbits is part of each run's protocol and must be archived with the base (Section 6.1).